

Managing Experiments

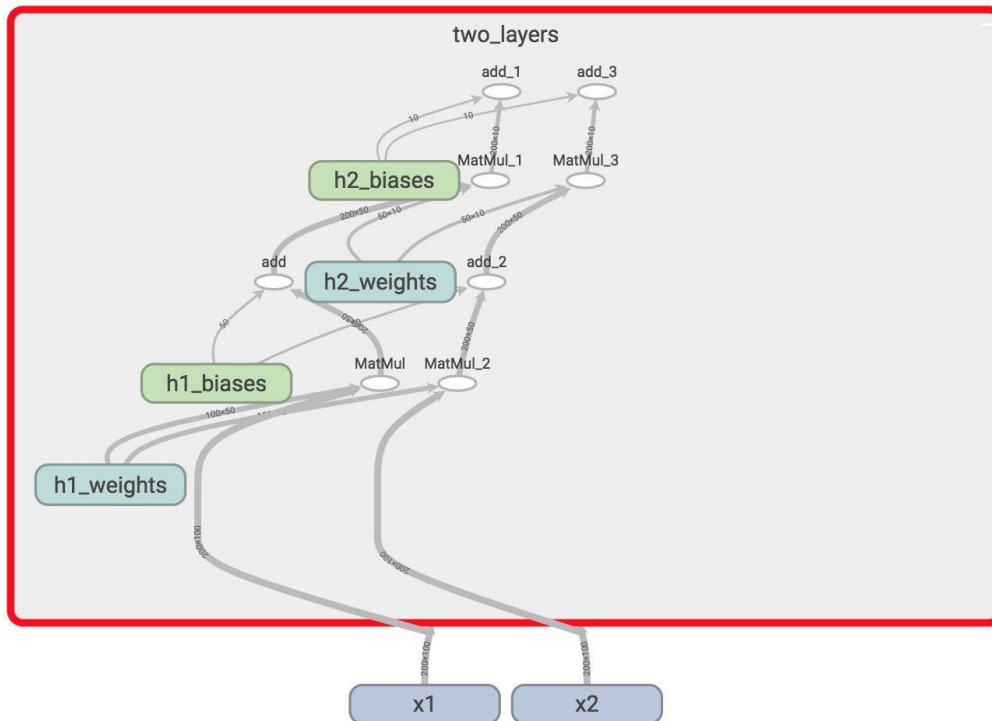
Each time you call two network, TensorFlow creates a different set of variables, while in fact, you want the network to share the same variables for all inputs, whether it's x1, x2, or more. To do this, we first need to use `tf.get_variable()`. When we create a variable with `tf.get_variable()`, it first checks whether that variable exists. If it does, reuse it. If not, create a new one. However, if we simply replace `tf.Variable()` with `tf.get_variable()` such as the following:

```
def two_hidden_layers_2(x):  
    assert x.shape.as_list() == [200, 100]  
    w1 = tf.get_variable("h1_weights", [100, 50], initializer=tf.random_normal_initializer())  
    b1 = tf.get_variable("h1_biases", [50], initializer=tf.constant_initializer(0.0))  
    h1 = tf.matmul(x, w1) + b1  
    assert h1.shape.as_list() == [200, 50]  
    w2 = tf.get_variable("h2_weights", [50, 10], initializer=tf.random_normal_initializer())  
    b2 = tf.get_variable("h2_biases", [10], initializer=tf.constant_initializer(0.0))  
    logits = tf.matmul(h1, w2) + b2  
    return logits
```

We will run into this issue:

To avoid this, we need to put all variables we want to use in a `VarScope`, and set that `VarScope` to be reusable.

Let's look at TensorBoard.



There's only one set of variables now, all within the variable_scope `two_layers`. They take in two different inputs `x1` and `x2`. `tf.variable_scope("name")` implicitly opens a `tf.name_scope("name")`.

In our code, we write code for each layer. When we have more layers that are similar in structure, we probably want to make our code more reusable.

You can scale it to infinitely many layers with different kinds of activation functions!

Graph collections

As you create a model, you might put your variables to different parts of the graph. Sometimes, you'd want an easy way to access them. `tf.get_collection` lets you access a certain collection of variables, with `key` being the name of the collection, `scope` is the scope of the variables.

By default, all variables are placed in `tf.GraphKeys.GLOBAL_VARIABLES`. To get all variables in scope `"my_scope"`, simply call. This turns a list of variables in `"my_scope"`.

If you set `trainable=True` (which is always set by default) when you create your variable, that variable will be in the collection `tf.GraphKeys.TRAINABLE_VARIABLES`.

You can have collections of ops that aren't variables. And yes, you can create your own collections with `tf.add_to_collection(name, value)`. For example, you can create a collection of initializers and add all init ops to that.

The standard library uses various well-known names to collect and retrieve values associated with a graph. For example, the `tf.train.Optimizer` subclasses default to optimizing the variables collected under `tf.GraphKeys.TRAINABLE_VARIABLES` if none is specified, but it is also possible to pass an explicit list of variables. For the list of predefined graph keys, please see [the official documentation](#).

Manage your experiments

We've built a word2vec model and it seems to be working pretty well using such a small dataset. We know that it'd take much longer time for a larger dataset, and we also know that training more complicated models can take an ungodly amount of time. For example, a machine translation models can take up to days, if not months on a single GPU. Many computer vision and reinforcement learning tasks require a really long time and a lot of patience.

It's difficult to let our models run for days, wait and then make adjustment. If the computer or the cluster crashes, the training is interrupted and we'll have to run our model all over again! It's crucial to be able to stop training at any point, for any reason, and resume training as if nothing happens. It will be helpful for analyzing our models, as this allows us closely inspect our models after any number of training steps.

Another problem that researchers often face is how to replicate research results. When building and training neural networks, we often use randomization. For example, we randomize the weights for our models, or we shuffle the order of our training samples. It's important to learn how to control this random factor in our models.

In this part of the lecture, we will go over the excellent set of tools that TensorFlow provides to help us manage our experiments, including but not limited to `tf.train.Saver()` class, TensorFlow's random state, and visualization our training progress (aka more TensorBoard).

`tf.train.Saver()`

A good practice is to periodically save the model's parameters after a certain number of steps or epochs so that we can restore/retrain our model from that step if need be. The `tf.train.Saver()` class allows us to do so by saving the graph's variables in binary files.

For example, if we want to save the variables of the graph after every 2000 training steps, we do the following:

In TensorFlow lingo, the step at which you save your graph's variables is called a checkpoint. Since we will be creating many checkpoints, it's helpful to append the number of training steps our model has gone through a variable called `global_step`. It's a variable you'd see in many TensorFlow programs. We first need to create it, initialize it to 0 and set it to be not trainable, since we don't want TensorFlow to optimize it.

We need to pass `global_step` as a parameter to the optimizer so it knows to increment `global_step` by one with each training step.

To save the current values of variables in the folder 'checkpoints', we use this:

To restore the variables, we use `tf.train.Saver.restore(sess, save_path)`. For example, to restore the checkpoint at the 10,000th step.

But of course, we can only load saved variables if there is a valid checkpoint. What you probably want to do is that if there is a checkpoint, restore it. If there isn't, train from the start. TensorFlow allows you to get checkpoint from a directory with `tf.train.get_checkpoint_state('directory-name')`. The code for checking looks something like this:

The file checkpoint automatically keeps track of the path to the latest checkpoint, so if we find the latest checkpoint from , `'checkpoints/checkpoint'` we can always get the latest checkpoint. This is an example of what the file `'checkpoints/checkpoint'` looks like.

So our training loop for word2vec now looks like that:

If you go to the folder 'checkpoints', you will see files like the below:

By default, `saver.save()` stores all variables of the graph, and this is recommended. However, you can also choose what variables to store by passing them in as a list or a dict when we create the saver object. This is an example from TensorFlow official documentation.

Note that savers only save variables, not the entire graph, so we still have to create the graph ourselves, and then load in variables. The checkpoints specify the way to map from variable names to tensors.

What people usually do is not just save the parameters from the last iteration, but also save the parameters that give the best result so far so that you can evaluate your model on the best parameters so far.

`tf.summary`

We've been using matplotlib to visualize our losses and accuracy, which is unnecessary because TensorBoard provides us with a great set of tools to visualize our summary statistics during our training. Some popular statistics to visualize is loss, average loss, accuracy. You can visualize them as scalar plots, histograms, or even images. So we have a new namespace in our graph to hold all the summary ops.

Because it's an op, you have to execute it with `sess.run()`

Now you've obtained the summary, you need to write the summary to file using the same `FileWriter` object we created to visualize our graph.